

Machine Learning (2025)

Homework 3: CNN

Deadline: 2025/5/14

Use CNN model to do the classification of the facial images, which are classified into 7 different expressions with size restricted to 48x48, including Angry, Disgust, Fear, Happy, Sad, Surprise, Neutral.

For example:

Angry:



Disgust:



Happy:



Prediction Script (predict.py) Requirements

You must submit a Python script named **predict.py** that allows (TAs) to evaluate your trained model on unseen test data.

The entire development process must be demonstrated in Google Colab.

Execution:

TAs will run your script from the command line using the following format:

```
$ python predict.py model.pth test output.csv
```

- **model.pth**: This is the weights file where your trained model is saved.
- **test**: This is the folder that will contain the test images for evaluation.
- **output.csv**: This is the file where the prediction results will be saved.

Functionality:

- The script must load the trained model specified by **<model.pth>**
- It must iterate through all image files directly inside the **<test>**. It's recommended to process images in alphanumeric order of their filenames for consistency.
- For each image, it must predict the emotion based on your trained model.
- The predicted emotion must be mapped to an integer according to the following convention:

- 0 : Angry
- 1 : Disgust
- 2 : Fear
- 3 : Happy
- 4 : Sad
- 5 : Surprise
- 6 : Neutral

Output.csv Format example:

	A
1	0
2	1
3	2
4	3
5	3
6	5
7	6
8	2
9	2
10	1
11	1

Homework Rules and Grading Policy

Homework will be graded by:

1. **Baseline Performance on Test Data (25%):** Achieve a test accuracy greater than 66% to receive points for this section.
2. **Performance rank(25%)**
3. **Report (50%):**
 - Describe your model architecture details.(25%)
 - Specify the loss function and plot training loss curves.(10%)
 - Discuss your approach to data imbalance in training.(15%)

Remind:

- **Program:** Please implement your solution **using Python and the PyTorch framework..** Training must be performed **without using pre-trained weights.** (training your model from scratch)
- **Report:** Use font size **12**, with a maximum of **7** pages. Explaining the key aspects of your approaches and visualizing your experimental results will earn a higher score on the report.
- **Deadline:** Late submissions within **3** days receive 70% of the score. Submissions beyond **3** days are NOT accepted.
- **Plagiarism:** If two codes show high similarity, both will receive zero marks after confirmation.

Upload:

[Web] E3

[File Name] hw3_{student id}.zip

(e.g., hw3_123456789.zip)

```
hw3_{student id}/
├── test/
├── output.csv
├── report.pdf
├── src/
│   ├── train.py
│   └── ...
├── predict.py
├── model.pth
└── README.txt
```

References on Imbalanced Data

- DeVries, T., & Taylor, G. W. (2017). *Improved Regularization of Convolutional Neural Networks with Cutout*.
- Zhang, H., Cisse, M., Dauphin, Y. N., & Lopez-Paz, D. (2017). *mixup: Beyond Empirical Risk Minimization*.
- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). *SMOTE: Synthetic Minority Over-sampling Technique*. Journal of artificial intelligence research, 16, 321-357.
- Han, H., Wang, W.-Y. and Mao, B.-H. (2005) ‘Borderline-SMOTE: A New Over-Sampling Method in Imbalanced Data Sets Learning’, in Huang, D.-S., Zhang, X.-P., and Huang, G.-B. (eds) *Advances in Intelligent Computing*. Berlin, Heidelberg: Springer Berlin Heidelberg, pp. 878–887.
- He, H. et al. (2008) ‘ADASYN: Adaptive synthetic sampling approach for imbalanced learning’, in 2008 IEEE international joint conference on neural networks (IEEE world congress on computational intelligence). IEEE, pp. 1322–1328.
- Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., ... & Bengio, Y. (2014). *Generative adversarial nets*. Advances in neural information processing systems, 27.
- Lin, T. Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). *Focal loss for dense object detection*. In Proceedings of the IEEE international conference on computer vision (pp. 2980-2988).